Chapitre 1 : Introduction à l'Orchestration et à Kubernetes

Cours : Cloud Computing & DevOps

Pr. Abdelfattah Abassi

FST Settat – Université Hassan Ier

28 octobre 2026

Plan du chapitre

1. Introduction et Objectifs
2. Cas d'étude : L'Application 3-Tiers
3. Limites de Docker Compose en Production
4. Concepts d'Orchestration & Philosophie
5. Architecture Globale de Kubernetes
6. Les Abstractions Fondamentales de Kubernetes
7. Comparatif : Docker Compose vs Kubernetes
8. Synthèse et Questions de Compréhension

Objectifs pédagogiques du chapitre

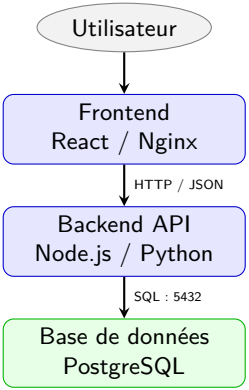
À la fin de ce chapitre, vous serez capable de :

- Comprendre la transition entre Docker Compose et un environnement distribué.
- Identifier les limites d'un déploiement sans orchestrateur.
- Définir l'orchestration de conteneurs.
- Expliquer le modèle déclaratif : état souhaité et état réel.
- Décrire l'architecture d'un cluster Kubernetes.
- Distinguer les notions de Pod, Deployment, ReplicaSet et Service.
- Comparer Docker Compose et Kubernetes.

Application modèle : Architecture 3-Tiers

Nous suivrons une application concrète tout au long du cours.

- **Frontend :** interface utilisateur React servie par Nginx.
- **Backend API :** API REST en Node.js ou Python.
- **Database :** base de données relationnelle PostgreSQL.



Architecture 3-Tiers

Gestion actuelle : Docker Compose

Avec Docker Compose, plusieurs conteneurs peuvent être exécutés et reliés sur une même machine.

```
services:
db:
image: postgres:15-alpine
environment:
POSTGRES_PASSWORD: secretpassword
volumes:
- db_data:/var/lib/postgresql/data

api:
image: myrepo/api-app:v1.0
environment:
DB_HOST: db
ports:
- "3000:3000"

web:
image: myrepo/frontend-app:v1.0
ports:
- "80:80"

volumes:
db_data:
```

Listing 1 – Extrait de docker-compose.yml

Pourquoi Docker Compose ne suffit-il pas toujours en production ?

1. Dépendance à une seule machine

Dans un déploiement Compose classique, si l'hôte tombe en panne, tous les services hébergés sur cette machine peuvent devenir indisponibles.

2. Résilience limitée à l'hôte

Une politique de redémarrage peut relancer un conteneur, mais elle ne déplace pas automatiquement l'application vers un autre serveur physique.

3. Gestion des mises à jour

Les mises à jour et le maintien de la disponibilité nécessitent une stratégie adaptée. Compose ne fournit pas, à lui seul, les mécanismes de déploiement distribué de Kubernetes.

Défis du passage à l'échelle et de la distribution

Exigences de production :

- Distribuer les applications sur plusieurs serveurs.
- Répartir le trafic entre plusieurs répliques.
- Adapter le nombre de répliques à la charge.
- Gérer le stockage persistant.
- Réagir aux pannes d'un serveur.

Questions à résoudre

- Où placer un nouveau conteneur API ?
- Que faire si un serveur tombe en panne ?
- Comment remplacer une version sans interruption ?
- Comment maintenir le nombre souhaité de répliques ?

Un orchestrateur automatise une grande partie de ces opérations.

Qu'est-ce que l'orchestration de conteneurs ?

Définition

L'**orchestration de conteneurs** consiste à automatiser leur déploiement, leur placement sur les machines, leur mise à l'échelle, leur communication réseau et la surveillance de leur état dans un cluster.

Principales responsabilités :

1. **Planification :** choisir le nud d'exécution.
2. **Auto-guérison :** réagir aux défaillances.
3. **Mise à l'échelle :** augmenter ou réduire les répliques.
4. **Mise à jour :** déployer progressivement de nouvelles versions.
5. **Réseau :** permettre la découverte et l'accès aux services.
6. **Stockage :** connecter les applications aux volumes nécessaires.

Paradigme déclaratif et boucle de réconciliation

Kubernetes utilise principalement une approche déclarative.

Mode impératif

On indique les actions à exécuter.

Exemple :

`Créer trois répliques de l'API.`

Mode déclaratif

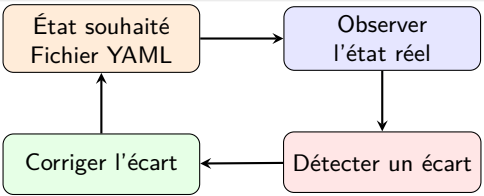
On décrit l'état que l'on souhaite obtenir.

Exemple :

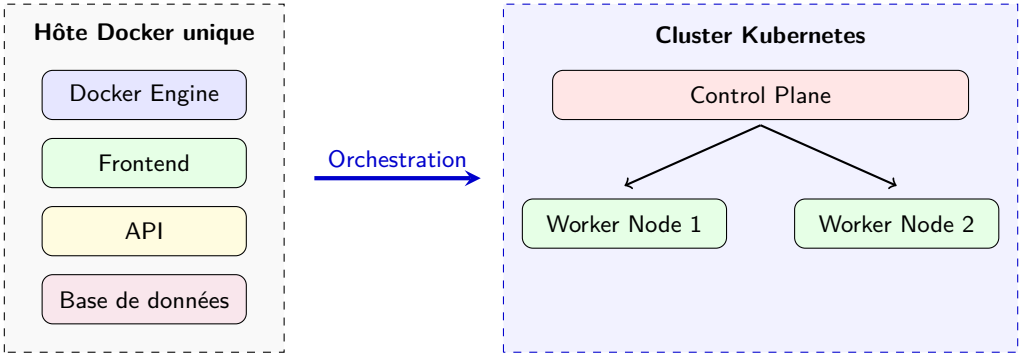
`Maintenir trois répliques de l'API.`

Boucle de réconciliation

Kubernetes compare continuellement l'état réel à l'état souhaité et agit lorsque les deux diffèrent.



Du serveur unique au cluster Kubernetes



Le cluster regroupe les machines et les composants nécessaires à la gestion et à l'exécution des applications.

Structure d'un cluster : Control Plane et Worker Nodes

1. Control Plane

Il assure la gestion du cluster :

- Expose l'API Kubernetes.
- Enregistre l'état des ressources.
- Planifie les Pods.
- Coordonne les contrôleurs.

2. Worker Nodes

Ce sont les machines qui exécutent les applications :

- Hébergent les Pods.
- Exécutent les conteneurs.
- Fournissent les ressources CPU et mémoire.
- Assurent les fonctions locales nécessaires au réseau.

Les composants internes du Control Plane

`kube-apiserver` Point d'entrée de l'API Kubernetes. Il reçoit les requêtes et permet aux composants de communiquer avec le cluster.

`etcd` Base de données clé-valeur qui stocke les données persistantes de l'état du cluster.

`kube-scheduler` Choisit un nud approprié pour les Pods qui ne sont pas encore affectés à un nud.

`kube-controller-manager` Exécute les contrôleurs qui observent les ressources et contribuent à rapprocher l'état réel de l'état souhaité.

Les composants sur chaque Worker Node

`kubelet` Agent local qui communique avec le Control Plane et veille à ce que les conteneurs des Pods soient exécutés conformément à leur configuration.

`kube-proxy` Composant réseau traditionnel qui configure des règles permettant d'acheminer le trafic des Services. Certains environnements utilisent d'autres implémentations réseau.

Container Runtime Logiciel chargé d'exécuter les conteneurs, par exemple `containerd` ou `CRI-O`.

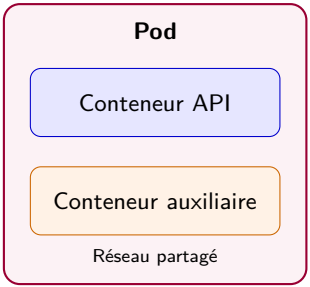
L'unité élémentaire : le Pod

Règle importante

Kubernetes planifie des Pods. Un Pod contient généralement un conteneur applicatif, mais peut en contenir plusieurs lorsqu'ils doivent fonctionner étroitement ensemble.

Caractéristiques d'un Pod :

- Plus petite unité déployable de Kubernetes.
- Contient un ou plusieurs conteneurs.
- Les conteneurs partagent le même espace réseau.
- Ils peuvent communiquer via `localhost`.
- Ils peuvent partager des volumes.
- Le Pod peut être recréé avec une nouvelle adresse IP.



Mise à l'échelle : ReplicaSet et Deployment

1. ReplicaSet

Maintient un nombre souhaité de Pods correspondant à un ensemble de critères. Si une réplique disparaît, il cherche à en créer une autre.

2. Deployment

Gère les mises à jour déclaratives d'applications sans état et supervise les ReplicaSets associés. Il permet notamment :

- De définir le nombre de répliques souhaité.
- De réaliser des mises à jour progressives.
- De revenir à une révision précédente.

À retenir

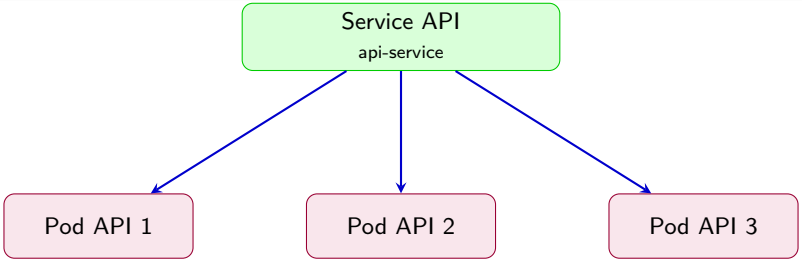
En pratique, on crée généralement un Deployment plutôt que de gérer manuellement les Pods et les ReplicaSets.

Exposition et stabilité réseau : le Service

Problème : les Pods peuvent être supprimés et recréés. Leurs adresses IP ne constituent donc pas des points d'accès durables pour les autres applications.

Solution : Service Kubernetes

Un Service fournit un point d'accès réseau stable et dirige le trafic vers les Pods correspondant à ses sélecteurs. Il peut également être découvert par un nom DNS interne.



Le trafic est dirigé vers les Pods disponibles correspondant aux sélecteurs du Service.

Configurations, environnements et stockage

Namespace Permet de regrouper et d'isoler logiquement des ressources dans un cluster, par exemple pour les environnements de développement et de production.

ConfigMap Stocke des paramètres de configuration non confidentiels.

Secret Permet de gérer des données sensibles. Son utilisation ne garantit pas à elle seule le chiffrement au repos : la configuration du cluster doit être vérifiée.

PersistentVolume (PV) et PersistentVolumeClaim (PVC) Permettent de représenter et de demander du stockage persistant indépendamment du cycle de vie d'un Pod.

Docker Compose vs Kubernetes

Critère	Docker Compose	Kubernetes
Portée	Généralement une machine	Cluster pouvant comporter plusieurs nuds
Objectif courant	Développement local et applications simples	Orchestration d'applications sur un cluster
Unité gérée	Conteneur et service Compose	Pod et ressources Kubernetes
Gestion des pannes	Redémarrage local selon la configuration	Contrôleurs de réconciliation et remplacement des Pods
Mise à l'échelle	Réplication généralement configurée manuellement	Réplication déclarative et autoscaling possible
Mises à jour	Stratégie dépendant de l'organisation du déploiement	Déploiements progressifs et retour arrière possibles
Complexité	Plus simple à prendre en main	Architecture et exploitation plus complexes

Attention

Kubernetes ne garantit pas automatiquement une disponibilité sans interruption. Celle-ci dépend aussi de la configuration, des sondes de santé, du nombre de répliques, du stockage et de l'infrastructure.

Synthèse du chapitre

Points essentiels à retenir

1. Docker Compose facilite le déploiement de plusieurs services, notamment en développement.
2. Kubernetes orchestre les applications dans un cluster.
3. Le modèle déclaratif permet de décrire l'état souhaité.
4. Les contrôleurs rapprochent l'état réel de l'état souhaité.
5. Le Control Plane gère le cluster et les Worker Nodes exécutent les Pods.
6. Le Pod est l'unité déployable de base.
7. Le Deployment gère les révisions et les mises à jour.
8. Le Service fournit un point d'accès réseau stable.
9. Les ConfigMaps, Secrets et ressources de stockage complètent les mécanismes de configuration et de persistance.

Questions de compréhension

Auto-évaluation

1. Pourquoi utilise-t-on généralement des objets de gestion comme les Deployments plutôt que de créer manuellement chaque Pod ?
2. Quelle est la différence entre l'état souhaité et l'état réel ?
3. Quel est le rôle du kube-scheduler ?
4. À quoi servent etcd et kube-apiserver ?
5. Pourquoi ne faut-il pas utiliser directement l'adresse IP d'un Pod comme adresse permanente de l'API ?
6. Quelle différence existe-t-il entre un Pod, un ReplicaSet et un Deployment ?
7. Pourquoi un Service est-il utile lorsque les Pods sont recréés ?
8. Kubernetes garantit-il automatiquement l'absence totale d'interruption pendant une mise à jour ? Justifiez votre réponse.